Desktop Employee Time & Activity Tracker

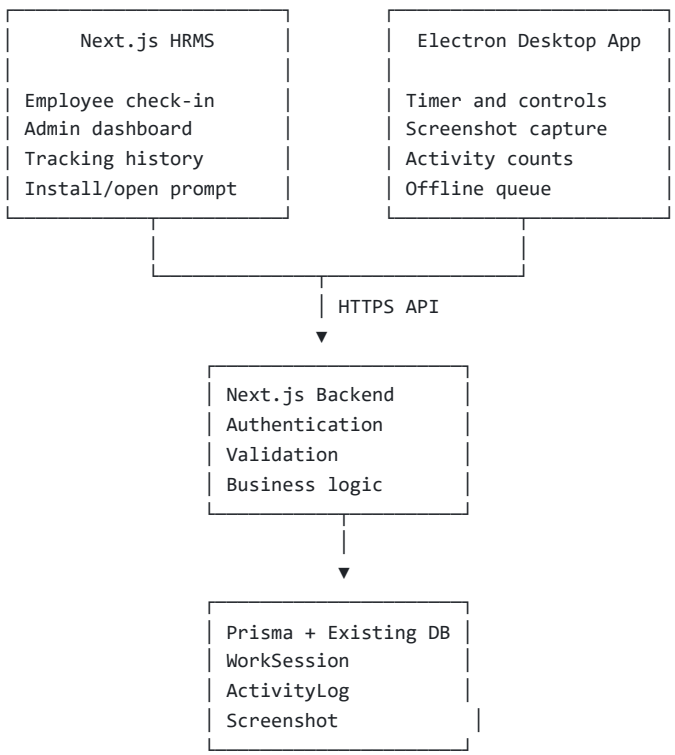
Technical Plan — Version 2

> **Goal:** Integrate a desktop time and activity tracker with the existing Next.js HRMS. Work sessions, activity summaries, and screenshots will be stored through the existing backend and database, then displayed in the HRMS dashboard.

1. Decisions at a Glance

Area	Decision
Desktop app	Electron + React + TypeScript
Existing web app	Next.js on Vercel
Backend and database access	Existing backend API + Prisma
Database	Reuse the existing database
Tables	`WorkSession`, `ActivityLog`, `Screenshot`
Offline recovery	SQLite queue + local screenshot files
Screenshot retention	Delete screenshots older than 15 days
Activity tracking	Mouse-click and keyboard-event counts only
Heartbeat	Not required in the first version
Missing desktop app	Show an installation prompt and notify the admin if tracking is required

2. System Architecture



Important: The desktop app never connects directly to the database and never contains database credentials.

Both the web app and desktop app communicate with the same backend and use the same database.

3. Employee Flow

1. Employee signs in to the HRMS.
2. Employee checks in for work.
3. The HRMS checks whether the desktop tracker is installed or has recently connected.
4. If the app is missing, the HRMS displays a **Download Desktop Tracker** prompt.
5. If installed, the employee opens the desktop app and signs in using their existing HRMS account.
6. The desktop app authenticates through the existing backend.
7. The backend verifies the employee's identity and checks whether tracking is permitted for their current session.
8. The desktop app captures permitted tracking data and uploads it through the backend.
9. The admin can view sessions, screenshots, activity summaries, and tracker status in the HRMS dashboard.

3.1 How desktop authentication works

- The employee uses the same account as the HRMS.
- The desktop app sends login details to a secure backend authentication endpoint.
- The backend validates the credentials using the existing authentication system.
- The backend issues an appropriate session or access token.
- The desktop app securely stores the authentication state and includes it in subsequent API requests.
- The backend verifies the token and employee permissions before accepting data.

Implementation note: The exact approach depends on the authentication library already used by the HRMS. Reuse that system rather than creating a second account system.

A future improvement could let employees open the desktop app from the HRMS website using a short-lived, single-use authorization code.

4. Desktop App Controls and Background Behavior

4.1 Employee controls

The desktop app should provide:

- Sign in and sign out.
- Start, pause, resume, and stop tracking.
- Elapsed tracked-time timer.
- Tracking status: active, paused, offline, syncing, or permission issue.
- Pending-upload count.
- Last successful sync time.
- Automatic recovery of pending data after a restart.
- A clear, visible indicator while tracking is active.
- also view the screenshots activity.

4.2 What happens when the desktop app is closed?

Closing the app window and quitting the app should be treated differently.

Recommended behavior:

- **Explicit Quit/Exit:** Clearly explain that tracking will stop, then stop tracking or mark the session interrupted as appropriate.

The exact tray behavior and startup settings must be tested on the target operating systems. Background tracking cannot be guaranteed if the operating system terminates the process or denies required permissions.

5. What Gets Tracked?

Data	How it is collected
Work time	Session start, end, pause, and resume timestamps
Screenshots	Randomized interval, initially around 8–12 minutes
Mouse activity	Mouse-click counts per minute
Keyboard activity	Keyboard-event counts per minute
Last observed activity	Timestamp of the latest permitted input event
Sync state	Pending, uploaded, or failed
Last sync	Time when the server last received data

Privacy rule: Never record actual typed characters, passwords, or keystroke content. Only store event counts.

6. Database Design

Use the **existing Prisma schema and database**. Do not create a separate database or duplicate employee records.

Reuse existing employee/user and attendance/check-in models wherever possible.

6.1 `WorkSession`

Stores one record for each desktop tracking session.

Suggested fields:

- `id`
- `employeeId` — relation to the existing employee/user model
- `startedAt`
- `endedAt`
- `status` — `ACTIVE`, `PAUSED`, `COMPLETED`, or `INTERRUPTED`
- `lastActivityAt`
- `lastSyncAt`
- `createdAt`
- `updatedAt`

6.2 `ActivityLog`

Stores periodic activity summaries.

Suggested fields:

- `id`
- `sessionId` — relation to `WorkSession`

- `intervalStart`
- `intervalEnd`
- `mouseClickCount`
- `keyboardEventCount`
- `status` — `ACTIVE`, `INACTIVE`, or `UNTRACKED`
- Stable unique record ID or idempotency key to prevent duplicate uploads

6.3 `Screenshot`

Stores screenshot metadata and the image itself.

Suggested fields:

- `id`
- `sessionId` — relation to `WorkSession`
- `capturedAt`
- `imageData` — binary image data stored in the database
- Stable unique record ID or idempotency key to prevent duplicate uploads

Why use three tables?

Each table has a distinct purpose:

- `WorkSession` stores the overall tracking session.
- `ActivityLog` stores many activity intervals within a session.
- `Screenshot` stores many screenshots within a session.

Putting everything into one table would duplicate session information and create many empty fields. Separate connected tables make querying, maintaining relationships, and deleting old screenshots easier.

Database notes:

- Connect `ActivityLog` and `Screenshot` to `WorkSession` through Prisma relations.
- Store screenshots in the database as requested, while monitoring database size, upload limits, and performance.
- Delete screenshots older than 15 days. Keep work-session and activity history according to the HRMS retention policy.

7. HRMS Dashboard

7.1 Employee view

- Attendance and check-in/check-out status.
- Current work-session status.
- Tracker status: not installed, connected/recently synced, offline, or permission issue.
- Pending-sync status.
- Permitted session and activity summaries.
- Installation instructions if the app is missing.

7.2 Admin/manager view

- Employee and work-session details.
- Session duration and status.
- Tracker status and last successful sync.

- Screenshots with capture timestamps.
- Mouse/keyboard activity summaries by minute.
- Active, inactive, and untracked intervals.
- Filters by employee, session, and date.
- Alerts when required tracking is missing.
- Alerts when local storage is nearing its limit.
- Interrupted sessions requiring review.

8. Offline Handling and Failure Recovery

Problem	Expected behavior
App not installed	Show installation prompt and notify admin if required
Internet disconnects	Save pending records locally and show offline status
Internet returns	Automatically retry uploads
App crashes or restarts	Recover the persistent local queue
Computer sleeps or shuts down	Preserve saved data and mark uncertain gaps as untracked/interrupted
Screenshot deadline is missed	Do not capture multiple screenshots to catch up
Duplicate upload	Use unique IDs/idempotency keys to prevent duplicate writes
Upload fails	Keep the local copy until the server confirms success
Local storage nearly full	Warn employee, notify admin when possible, and enforce safe limits
Permissions unavailable	Show a warning and mark the affected interval untracked
Tracker stops syncing	Show stale/not-connected status based on `lastSyncAt`; notify admin after a configured threshold

8.1 Why can local storage become nearly full?

as soon as the data is pushed into the database delete the locally stored data

Important: If the employee's computer is offline, the server cannot receive an immediate admin notification. The desktop app should queue the alert and send it when connectivity returns.

8.2 Local-first synchronization

```
Capture activity or screenshot
            ↓
Save locally
(SQLite + local screenshot file)
            ↓
Send to backend when online
            ↓
Backend validates and saves through Prisma
            ↓
Server confirms successful storage
            ↓
Remove the local pending copy
```

The local queue must survive app restarts. Remove pending files only after the backend confirms that the corresponding records have been stored successfully.

9. Security and Retention

- Authenticate desktop API requests.
- Verify employee and session ownership on every relevant request.
- Keep database credentials and server secrets on the server.
- Validate image types and upload sizes.
- Protect locally stored pending files and authentication credentials.
- Never store actual typed text, passwords, or keystroke content.
- Display a visible tracking indicator and provide appropriate employee notice.
- Delete screenshot records and their image data after 15 days.
- Consider backups and replicas: deleting a live database row does not necessarily remove copies from existing backups immediately.
- Verify Vercel request-size and execution limits for screenshot uploads. If direct uploads are unsuitable, use an appropriate ingestion endpoint or service while keeping the existing database as the permanent image store.

10. Technology Stack

Component	Technology
Desktop application	Electron, React, TypeScript
Screen capture	Electron desktop-capture APIs and required OS permissions
System-wide input activity	OS-specific input-event mechanism
Local pending queue	SQLite and local screenshot files
Web UI	Existing Next.js application
Backend and data access	Existing backend API and Prisma
Permanent storage	Existing database
Cleanup and retries	Scheduled job or worker